CONTRACT.md — the single source of truth for Ripple's data shapes

Every AI tool, every teammate, generates against what's in this file, not an earlier version of it. If your task needs a shape that isn't here, stop, propose the addition, get it added, then generate. This file is owned for edits by Lead.

Source: Aman's Track 2 engine spec, accepted as the starting contract with minor reformatting. Status: proposed first-version design for team alignment, not yet validated against every edge case.

---

1. What the engine answers

What can execute as planned? What gets blocked, and why? Which other commitments are affected? How do proposed responses change the outcome?

Models the operating cycle connecting supplier payments, material availability, production, delivery, and customer collections. A profitable order can still be impossible to fulfil on time if the business lacks cash at the necessary moment.

2. Scope, v1

Working example: a small garment workshop (or textile wholesaler, per Track 3's persona). Supports: multiple orders sharing cash and inventory; supplier payments and material arrivals; production and delivery dependencies; existing customer receivables; collections linked to future deliveries; explicit payment/delivery disruptions; baseline vs. disrupted vs. intervention scenario comparison; explanations for blocked or delayed commitments.

Simplifications for v1, don't build beyond these under time pressure: time in whole days; fixed production duration per order; whole-order deliveries only; payments execute in full (instalments are separate events); production capacity assumed sufficient; no borrowing/overdraft modeled. Partial deliveries, capacity scheduling, alternate sourcing, probabilistic forecasts are explicitly out of scope for v1.

3. Core model: events and dependencies

Event	Effect
Customer receipt	Adds cash
Supplier payment	Deducts cash
Material arrival	Adds inventory
Production start	Consumes required inventory and deducts defined upfront production cost
Production completion	Marks an order ready for delivery
Delivery	Marks the order delivered
Other expense	Deducts cash according to its own timing and prerequisites

An event executes only when: its earliest execution day has arrived, its required predecessor events have executed, any required delay after those events has elapsed, and sufficient cash and inventory are available.

Cash and inventory are shared across orders. A late collection can affect an order with no direct dependency on that customer, this is the mechanism the whole product demonstrates.

4. Production-cost rule

Production start requires both enough material and enough cash for the defined production-start cost. The engine deducts cash and consumes material together, atomically, if either is insufficient, neither is consumed. Fixed wages, rent, booking advances, and other expenses are represented separately by their own timing, they do not inherit this rule automatically.

5. Data contract

Business snapshot: business identifier, simulation start date, planning horizon, opening available cash, opening inventory quantities, scheduling policy.

Order: order identifier, customer identifier, promised delivery date, material requirements, production-start cost, production duration, scheduling priority.

Event: unique event identifier, event type, related order and counterparty (where applicable), earliest execution date, original due date (where applicable), dependencies and their time lags, cash amount or inventory movement (where applicable).

Scenario: scenario identifier, changes to baseline commitments, additional or modified events, assumptions and required agreements.

Result: daily cash and inventory balances, actual simulated execution dates, blocked and unresolved events, missed commitments and delivery slippage, blocking reasons and explanation traces, applied scheduling policy and assumptions.

Conventions: money in integer paise, every inventory item has a defined unit, dates use one agreed format. Baseline commitments, recorded actual transactions, and simulated outcomes stay in separate fields, never merged.

6. Timing rules

Two distinct behaviors: a **fixed expected date** (e.g. collection for goods already delivered) versus **relative to an event** (e.g. collection four days after actual delivery, so if delivery moves from Day 6 to Day 10, the collection moves from Day 10 to Day 14). Original due dates and promised delivery dates never change in the results, so slippage stays visible. Material availability is a resource condition, production doesn't depend on a specific future shipment if sufficient stock already exists.

7. Simulation approach

Deterministic daily simulation over a finite horizon. Each day: identify events whose date and prerequisites permit execution, check cash/inventory requirements, execute eligible events per the scheduling policy, update shared resources immediately, recheck newly-enabled events, stop when nothing further can execute that day, record balances and outstanding commitments. Each event executes at most once; blocked events are reconsidered on later days. Reject invalid dependencies and dependency cycles before simulation runs.

8. Scheduling policy, v1

When commitments compete for resources: owner-entered priority, then earliest original due date, then stable event ID as tie-breaker. The policy must be visible in the results. The engine simulates this policy, it does not claim to find the globally optimal schedule. An unresolved commitment at the horizon's end means it couldn't execute under the supplied assumptions and policy within that horizon, not that no arrangement could work.

9. Correctness checks, required before anyone trusts a demo run

Never spend or allocate resources twice. Never execute an event twice. Never allow negative executable cash or inventory. Preserve unchanged baseline data. Reduce remaining collections when introducing deposits. Recalculate delivery-relative receipts after delivery changes. Avoid propagating delays that existing resources can absorb. Produce reproducible results for identical inputs and policy. Explain every unresolved commitment.

Validation cases to run before trusting the engine in front of judges: healthy cycle, absorbed delay, blocked purchase, existing stock covers a gap, instalments, competing orders, refused intervention, unresolved commitment at horizon end.

10. What the frontend needs from this engine (from Track 3's spec)

Per-scenario daily balance series; first-negative-cash day; lowest balance and its date; buffer-breach date and amount; list of affected orders/parties; an **ordered causal trace** (step, amount, date, node id) for the "why" explanation screen; the chain graph itself (nodes, edges, due dates, lead times); P10/P50/P90 delay estimate per customer, if time allows a confidence layer on top of the deterministic core.

11. Open decisions before implementation (Aman's list, still open)

Target workflow confirmation. Final data fields and units. Production-cost timing edge cases. Same-day execution assumptions. Final scheduling policy. Which interventions v1 actually supports (deposit splitting is the only one worked through in the example).

12. Status flags

Everything in this file is **CONFIRMED as the team's own design decision**, not an external fact requiring verification, unlike the regulatory content in `PS_CONTEXT.md` section 9, which does need source-checking before judge-facing use.